However, a value that is too large will make sampling from the posterior GP numerically unstable. You could play around with multiple thousands of points to find the number that works best for your use case, objective function, and trained Gaussian process. However, you should always use at least one thousand points.

Moving on to the second component of TS, which is the process of sampling a multivariate Gaussian from our posterior GP and maximizing it. First, the sampler object that implements the sampling may be declared as:

```
ts = botorch.generation.MaxPosteriorSampling(model, replacement=False)
```

The `MaxPosteriorSampling` class in BoTorch implements the logic of TS: sampling from the posterior of a GP and maximizing that sample. Here, `model` refers to the GP that is trained on observed data. It is important to set `replacement` to `False`, making sure that we are sampling without replacement (sampling *with* replacement isn't appropriate for TS). Finally, to obtain the data point that gives the highest sampled value among `candidate_x`, we pass it to the sampler object:

```
next_x = ts(candidate_x, num_samples=1)
```

The returned value is indeed the point that maximizes the sample, which is the point we will query next. And with that, our implementation of the TS policy is complete. We may plug this in to the code we have been using so far for the BayesOpt loop on the Forrester function with:

```
for i in range(num_queries):
  print("iteration", i)
  print("incumbent", train_x[train_y.argmax()], train_y.max())

  sobol = torch.quasirandom.SobolEngine(1, scramble=True)   ❶
  candidate_x = sobol.draw(num_candidates)   ❶
  candidate_x = 10 * candidate_x - 5   ❷

  model, likelihood = fit_gp_model(train_x, train_y)

  ts = botorch.generation.MaxPosteriorSampling(model, replacement=False)   ❸
  next_x = ts(candidate_x, num_samples=1)   ❸

  visualize_gp_belief_and_policy(model, likelihood, next_x=next_x)   ❹

  next_y = forrester_1d(next_x)

  train_x = torch.cat([train_x, next_x])
  train_y = torch.cat([train_y, next_y])
```

❶ Generate points from a Sobol engine.

❷ Resize the generated points to be between -5 and 5, which is our search space.

❸ Generate the Thompson sampling candidate to be queried next.

❹ Visualize our current progress without the acquisition function.

Note that the overall structure of our BayesOpt loop remains the same. What's different is that,

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>